

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL
CARRERA DE INGENIERÍA DE SOFTWARE

APLICACIONES WEB – 5TO NIVEL – PARALELO A

TEMA:

Informe Técnico del Examen Unidad IV

**Modelo Vista Controlador y Servicios Web: API REST, Autenticación JWT
y Consumo de APIs Externas**

ESTUDIANTE: IRVIN MARCELO CAJAS IBARRA
DOCENTE: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
REPOSICION: https://github.com/Theirvin1/examen_appweb
PERIODO: 2026–2027 PPA

1 Introducción y Objetivos

Este informe describe la solución técnica desarrollada para la evaluación práctica de la **Unidad IV** en la asignatura Aplicaciones Web. Partiendo del proyecto base `biblioteca-u4-base`, se extendió la aplicación para exponer una API REST completa, construida sobre **Java 25** y **Spring Boot 4.1.1**, siguiendo los principios del estilo arquitectónico REST propuestos originalmente por Fielding [1].

El trabajo realizado cubrió los siguientes puntos:

- Diseño de controladores REST con filtros dinámicos, paginación y una estructura de respuesta uniforme mediante el envoltorio `ApiResponse`, en línea con las prácticas de diseño analizadas por Neumann et al. sobre APIs REST públicas [2].
- Mecanismo de autenticación sin estado basado en JWT (*JSON Web Token*), con autorización por roles para proteger los endpoints sensibles, siguiendo el esquema de sesión con HMAC-SHA512 descrito por Dalimunthe et al. [3].
- Consumo de la API externa OpenLibrary aplicando el patrón *Cache-Aside* sobre Redis 8, con una estrategia de invalidación acorde a lo propuesto por Falkevych y Lisniak para arquitecturas de microservicios [4].
- Suite de pruebas de integración automatizadas con Testcontainers 2.x y PostgreSQL 18.

2 Matriz Tecnológica y Entorno

Table 1: Componentes de software y versiones utilizadas.

Componente	Versión	Función / Observación
Java	25 (LTS)	Lenguaje base (SDK 25)
Spring Boot	4.1.1	Framework principal (<code>spring-boot-starter-webmvc</code>)
Spring Data JPA	4.1.x	Persistencia y consultas dinámicas mediante Specifications
Flyway	12.4.0	Migración de esquemas SQL (V1, V2, V3)
PostgreSQL	18	Base de datos relacional en Docker
Redis	8	Memoria caché (<i>Cache-Aside</i>)
Testcontainers	2.0.5	Entorno aislado para pruebas de integración

3 Desarrollo Técnico

3.1 B1. Endpoints REST y Envoltorio Estándar

Se creó el controlador `LibroController` para administrar las operaciones sobre los libros del sistema. El endpoint `GET /api/v1/libros` admite parámetros opcionales de búsqueda (`titulo`, `categoriaId`, `anioDesde`), los cuales son procesados por `LibroService.buscar(...)`. Los resultados se entregan envueltos en el objeto `ApiResponse`, junto con la información de paginación contenida en `PageMeta`. Este diseño responde a las observaciones de Neumann, Laranjeiro y Bernardino sobre la heterogeneidad de adherencia a los principios REST en APIs públicas reales [2], buscando así una interfaz consistente y predecible para los consumidores del servicio.

3.2 B2. Autenticación JWT y Seguridad por Roles

El esquema de autenticación no mantiene estado en el servidor. El controlador `AuthController` valida las credenciales del usuario y devuelve un token JWT firmado con el algoritmo HMAC-SHA. Cada petición entrante pasa por el filtro `JwtAuthenticationFilter`, encargado de reconstruir el contexto de seguridad a partir del token recibido. Las rutas de escritura, como `POST /api/v1/libros`,

quedan restringidas mediante la anotación `@PreAuthorize("hasRole('ADMIN')")`. Este enfoque de gestión de sesión sin estado, firmado con HMAC-SHA512, sigue el modelo evaluado por Dalimunthe, Putra y Ridha para servicios RESTful [3].

3.3 B3. API Externa con Cache-Aside y Manejo de Fallos

Se implementó el cliente HTTP `OpenLibraryClient` para enriquecer los registros de libros con metadatos adicionales obtenidos por ISBN.

- **Caché:** los resultados se almacenan en el espacio de caché `openlibrary`, con una expiración (TTL) de 24 horas en Redis.
- **Resiliencia:** si el proveedor externo responde con un código 404, los campos externos se devuelven como `null` sin afectar el código 200 OK de la respuesta local del libro. Ante fallos de red o errores 5xx, se lanza una `ServicioExternoException`, que el manejador global traduce automáticamente a un estado 502 `Bad Gateway`.

La estrategia de invalidación y separación de responsabilidades entre la lógica de negocio y las reglas de actualización de caché sigue el enfoque declarativo propuesto por Falkevych y Lisniak para arquitecturas de microservicios [4].

3.4 B4. Pruebas de Integración

En la clase `LibroControllerIT` se implementaron pruebas automatizadas ejecutadas contra una instancia real de PostgreSQL en contenedor:

1. Verificación de que el listado responde con estado 200 OK y la estructura esperada del envoltorio `ApiResponse`.
2. Comprobación de que una solicitud a un recurso inexistente devuelve un error 404 `Not Found` con formato `ProblemDetails`.
3. Validación de que un cuerpo de solicitud inválido produce una respuesta 400 `Bad Request`.

4 Comandos de Compilación del Informe

Para dar cumplimiento al criterio de evaluación obligatorio, la cadena de compilación de este documento desde la terminal en el directorio `docs/` es la siguiente:

```
cd docs
pdflatex informe.tex
bibtex informe
pdflatex informe.tex
pdflatex informe.tex
```

5 Conclusiones

Este ejercicio permitió consolidar el uso de una arquitectura en capas con Spring Boot 4.1.1 sobre Java 25. Se comprobó el valor de aplicar convenciones REST rigurosas de mantener la autenticación JWT desacoplada de la lógica de negocio mediante un esquema de firma robusto , y de aislar los servicios externos a través de una estrategia de caché bien definida, lo cual mejora la resiliencia general del sistema.

References

- [1] R. T. Fielding and R. N. Taylor, “Principled design of the modern web architecture,” *ACM Transactions on Internet Technology (TOIT)*, vol. 2, no. 2, pp. 115–150, 2002. DOI: <https://doi.org/10.1145/514183.514185>.
- [2] A. Neumann, N. Laranjeiro, and J. Bernardino, “An analysis of public rest web service apis,” *IEEE Transactions on Services Computing*, vol. 14, no. 4, pp. 957–970, 2021. DOI: <https://doi.org/10.1109/TSC.2018.2847344>.
- [3] S. Dalimunthe, H. E. Putra, and M. F. Ridha, “Restful api security using json web token (jwt) with hmac-sha512 algorithm in session management,” *IT Journal Research and Development*, vol. 8, no. 1, pp. 81–94, 2023. DOI: <https://doi.org/10.25299/itjrd.2023.12029>.
- [4] V. G. Falkeyvych and A. O. Lisniak, “Methodology of cache invalidation in microservices architecture of the web applications,” *Scientific Notes of Taurida National V.I. Vernadsky University, Series: Technical Sciences*, vol. 1, pp. 131–135, 2023. DOI: <https://doi.org/10.32782/2663-5941/2023.1/20>.